

PCAD
Programmazione Concorrente
Algoritmi Distribuiti

Arnaud Sangnier
arnaud.sangnier@unige.it

Problema del consensus

Il problema del consensus

- Dei processi/thread che comunicano tra loro (n processi $v_1, \dots, v_n$)
- Ciascun processo
 - Ha un valore in Input (diciamo un intero) x_i
 - Decidono un valore in Output y_i
 - Vogliono mettersi d'accordo sullo stesso valore in Output
- **Proprietà da verificare**
 - 1) **Agreement**
 - L'output di tutti i processi (che non falliscono) deve essere lo stesso
 - 2) **Validity**
 - L'output deve essere uno dei valori in Input
 - In particolare, se tutti i processi hanno lo stesso valore di Input x , tutti gli output sono uguali a x
 - 3) **Termination**
 - Tutti i processi (che non falliscono) terminano in un numero finito di passi

Perché il consensus è importante

- Se abbiamo un algoritmo per il problema del consensus, possiamo risolvere tanti altri problemi distribuiti
- Ad esempio:
 - Eleggere un leader
 - Risolvere la mutua esclusione
 - Risolvere il problema dei due generali
 - E tanti altri
- Sotto alcune ipotesi, il problema del consensus può essere risolto, ma in generale non è possibile (come il problema dei due generali non può essere risolto in comunicazione sincronizzata con perdita di messaggi)
- Uno degli obiettivi di chi si occupa di algoritmi distribuiti è capire quali ipotesi permettono di risolvere il problema del consensus

Soluzione #1

- Memoria condivisa e comunicazione asincrona
- La memoria condivisa può essere scritta/letta da tutti i processi
- I processi possono leggere o scrivere in modo atomico in una cella della memoria condivisa (ma non possono leggere e scrivere in modo atomico)
- Una possibile soluzione:

- `int c=?` (valore iniziale della cella condivisa `c`)
- Processi 1 scrive il suo input `x1` in `c` e decide `x1`
- Gli altri processi `j` con `j ≠ 1` leggono `c` finché il valore letto sia diverso da `?` e decidono il valore letto

- Quali sono i problemi di questa soluzione ?
 - Se il processo 1 è lento o fallisce non funziona

Qualche ipotesi in più

- Memoria condivisa con asynchrony
- Vogliamo una soluzione che funzioni anche in caso di crash fail di processi
- **Problema:** non possiamo sapere se un processo non reagisce perché è lento o perché è fallito...
- Vogliamo delle garanzie
- **Wait free algoritmi:**
 - Ogni processo finisce la sua esecuzione dopo un numero finito di passi (anche se tutti gli altri processi fanno un crash)
- Negli algoritmi wait free algoritmi, non possiamo usare i mutex
 - Lo thread che ha il mutex potrebbe andare in crash e gli altri processi si troverebbero bloccati

Un algoritmo wait free

```
int c=? //shared variable
--Process i (con input xi)
int r=0
r=c;
if(r== ?){
    c=xi;
    decide xi
}else {
    decide r;
}
```

- Questo algoritmo è wait free?
 - **Si**
- Questo algoritmo è corretto?
 - **No**

Esempio di esecuzione

Impossibilità

- **Teorema [FLP : Fischer, Lynch, Peterson ,1985]:**
 - Non esiste un algoritmo asincrono deterministico wait-free algoritmo per risolvere il problema del consensus con la memoria condivisa (e con atomica lettura e scrittura come uniche operazioni atomiche)
- Per dimostrarlo, consideriamo un contesto semplice
 - Solo due processi A e B con input in $\{0,1\}$ (input binari)
 - **Supponiamo l'esistenza di un algoritmo**
 - In questo algoritmo ad ogni momento, o A o B effettua un passo
 - Effettuare un passo=
 - Leggere un registro
 - Scrivere un registro

Albero di esecuzione

Valenza dei stati

- Nel albero di esecuzione, possiamo assegnare un'etichetta a ogni nodo con la sua «valenza »
 - Un nodo con **valenza 0** => in tutte le esecuzioni che partono da questo nodo, i processi decidono (un giorno) 0
 - Un nodo con **valenza 1** => in tutte le esecuzioni che partono da questo nodo, i processi decidono (un giorno) 1
 - Un nodo è **mono-valente** se ha valenza 0 o valenza 1
 - Un nodo è **bi-valente** se da questo ci sono delle esecuzioni in cui i processi decidono 0 e altre in cui decidono 1

Valenza dei stati

- Se i due processi iniziano con lo stesso valore, il nodo iniziale è per forza mono-valente
- Se i due processi iniziano con due valori diversi, il nodo iniziale è per forza bi-valente
 - Se A ha come input 0 e B ha come input 1 allora
 - Nell'esecuzione di A da solo, A deve decidere 0
 - Nell'esecuzione di B da solo, B deve decidere 1
- In un albero con un nodo bi-valente, ci deve essere un **nodo critico**:
 - Un nodo critico è bi-valente e i suoi due figli sono mono-valente (con valenze diverse)
 - Se non c'è un nodo critico, vuol dire che c'è un'esecuzione in cui si sta per sempre in un nodo bi-valente → Impossibile perché l'algoritmo è wait-free
 - Guardiamo le diverse azioni che partono da un nodo critico e troviamo una **contraddizione**

Azione dal nodo critico

Un nodo critico è bi-valente e i suoi due figli sono mono-valente (con valenze diverse)

Dal nodo critico possono essere possibili eseguire le seguenti azioni

1) A e B leggono un registro

Allora eseguire prima l'azione di A poi quella di B è come eseguire prima l'azione di B poi quella di A e si arriva allo stesso stato → **contraddizione**

2) A e B scrivono su due registri diversi

Allora eseguire prima l'azione di A poi quella di B è come eseguire prima l'azione di B poi quella di A e si arriva nello stesso stato → **contraddizione**

3) A e B scrivono sullo stesso registro

In questo caso le esecuzioni in cui A è da sola dal nodo critico e dove B fa un passo e poi A si esegue da sola devono portare alla stessa decisione per A

→ **contraddizione**

Quindi non esiste un algoritmo wait-free per il consensus

Consensus con altri ipotesi

- Si può fare consensus in sistemi distribuiti con message passing
- Se i messaggi vanno persi → NO (cf il problema dei due generali)
- Se i processi fanno un crash ?
 - In alcuni casi un crash può essere individuato:
 - Ad esempio con dei Timeout
- Si può fare il consensus in un sistema sincronizzato con message passing e con crash ?
- Modello :
 - Le comunicazione si fanno in round
 - Tutti i processi possono comunicare con tutti

Modello con broadcast e crash

- **Broadcast:** in un round, un processo può mandare un messaggio a tutti gli altri
 - Alla fine del round, tutti i processi hanno ricevuto il messaggio
 - Ogni processo può inviare un messaggio per round
- **Crash fail:**
 - Un broadcast può non essere mandato a tutti se il processo emittente fa un crash
 - Quindi alcuni messaggi possono essere persi

Un algoritmo semplice

- Ogni processo fa:

- 1) Broadcast il suo valore
- 2) Decide il valore minimo da tutti i valori ricevuti (incluso il suo valore)

- Questo algoritmo ha bisogno di un unico round
- Ma funziona se un processo fallisce ?
 - Se fallisce senza aver mandato il messaggio a nessuno → sì
 - Se fallisce dopo aver mandato il suo messaggio solo a una parte degli altri processi → no

Algoritmo per il consensus f-resilient

- Un modo per ottenere degli algoritmi consiste nell'aggiungere ipotesi sui fallimenti
- Un algoritmo per il consensus è **f-resilient** se è corretto nelle esecuzione con al massimo f processi che vanno un crash
- An algoritmo f-resilient per $f \leq n-2$:

- Round 1 :
Broadcast il suo valore
- Round 2 a $f+1$
Broadcast il valore minimo da quelli ricevuti (tranne se ha già mandato quel valore prima)
- Fine round $f+1$
Decide il valore minimo ricevuto

Algoritmo per il consensus f -resilient

- Round 1 :
Broadcast il suo valore
- Round 2 a $f+1$
Broadcast il valore minimo da quelli ricevuti (tranne se ha già mandato quel valore prima)
- Fine round $f+1$
Decide il valore minimo ricevuto

- Perché funziona:
 - Se ci sono f crash e $f+1$ rounds, ci sarà un round dove nessun processo fa un crash
 - Alla fine di questo round senza crash
 - Tutti i processi che sono vivi conoscono il valore di tutti gli altri processi
 - Questa conoscenza non cambia fino alla fine del algoritmo
 - Quindi alla fine decidono lo stesso valore